

Mock Design: A News Feed

Phase 4 · Interview-Ready · Estimated time: 2 hours

1. Concept From First Principles: Applying the Framework

A news feed (Facebook/Instagram-style) is a different shape of problem than chat: instead of delivering one message to one or a few recipients, you're generating a personalized, ranked stream for every user from potentially thousands of people they follow. The central architectural question — introduced briefly in Chapter 7's group chat fan-out — gets its full treatment here: **when** do you compute a user's feed, at post time or at read time?

Step 1 — Clarify requirements

- Functional: users follow other users; a feed shows recent posts from followed accounts, ranked (not strictly chronological, since engagement-based ranking is standard in real products); users can post text/images.
- Non-functional: feed load must feel instant (low read latency dominates the experience); the system must handle highly uneven follower distributions — most users have few followers, a small number of celebrity accounts have millions.

Step 2 — Estimate scale

Assume 300 million daily active users, each posting on average 0.5 times/day (~150 million posts/day, ~1,700 writes/second average) but each checking their feed ~10 times/day (~3 billion feed reads/day, ~35,000 reads/second average) — a read:write ratio over 1000:1, the single most important number in this entire design, since it justifies optimizing overwhelmingly for read performance even at real write-side cost.

Step 3 — API

POST /posts {userId, content} and GET /feed?userId=X&cursor=Y -> [posts...], using cursor-based pagination (not offset-based) since a feed is a constantly-growing, reverse-chronological-ish stream where offsets shift under you as new posts arrive.

Step 4 — High-level design

Client → Load Balancer → Feed Service, backed by a Post Store (durable, holds every post once) and a per-user precomputed Feed Store (holds each user's ready-to-serve feed, as a list of post IDs, typically cached in Redis for the most recent window). The critical question is how that precomputed Feed Store gets populated — which is this chapter's Step 5 deep dive in full.

Step 5 — Deep dive: fan-out on write vs. fan-out on read vs. hybrid

Fan-out on write (push model): when a user posts, the system immediately pushes that post into the precomputed Feed Store of every one of their followers, asynchronously via a message queue (the same pattern as Chapter 7's group chat fan-out). Reading a feed then becomes trivially fast: just read the already-built list for that user. This is excellent for the overwhelming majority of users with a normal

follower count — but breaks down catastrophically for a celebrity account with 50 million followers, since one post would require 50 million individual writes, a massive, slow, resource-heavy fan-out for a single post — this is the well-known **celebrity problem** / write-amplification problem.

Fan-out on read (pull model): nothing is precomputed at post time; instead, when a user requests their feed, the system fetches recent posts from everyone they follow, on demand, and merges/ranks them live. This avoids the celebrity write-amplification problem entirely, since a celebrity's post is written exactly once regardless of follower count — but makes every single feed read expensive, since it must fan out to query many followed accounts' recent posts live, which is a poor trade given this problem's 1000:1 read:write ratio favoring fast reads above nearly everything else.

Hybrid (the standard real-world answer): use fan-out on write for the vast majority of users (normal follower counts, cheap to push to), but for accounts above a follower-count threshold (say, 10,000+ followers), skip the push entirely and instead merge their posts into a follower's feed at read time. A user's final feed becomes: precomputed pushed posts (from normal accounts they follow) merged live with a small number of pulled posts (from any celebrity accounts they follow) — getting fast reads for the common case while avoiding write-amplification for the rare, expensive case. Naming this exact hybrid, with the specific reasoning for why neither pure approach works alone, is the single strongest signal in this entire chapter's deep dive.

Step 6 — Trade-offs and wrap-up

Volunteer: ranking (not strict chronology) adds real complexity — a full ranking model is out of scope for this interview, but naming that ranking signals (recency, engagement prediction, relationship strength) would typically be computed by a separate ranking service consuming the merged candidate post list shows awareness of the production-realistic next layer beyond just fan-out.

2. Real-World Example / Case Study

Meta's own publicly discussed feed infrastructure, and Twitter's/X's well-documented handling of exactly this celebrity fan-out problem (covered in more depth in Chapter 9), are the canonical real-world instances of the hybrid push/pull model — this is not a hypothetical interview trick but a genuinely necessary production pattern at any social platform with a highly skewed follower distribution.

3. Diagram

```

Normal user posts (few thousand followers):
[Post] -> [Post Store, write once]
          |
          [Message Queue]
        /   |   \
push to each follower's precomputed Feed Store
(fan-out ON WRITE)

Celebrity posts (50M followers):
[Post] -> [Post Store, write once]    <- NO fan-out push

Reading a feed (any user):
[GET /feed] -> merge:
  (a) precomputed pushed posts from normal follows
  (b) live-pulled recent posts from celebrity follows
          |
          [Feed Service] -> ranked, paginated feed

```

4. Trade-offs Table

Approach	Read cost	Write cost	Verdict
Fan-out on write (pure push)	Cheap (precomputed)	Catastrophic for celebrity accounts	Good for normal users only
Fan-out on read (pure pull)	Expensive (live merge every read)	Cheap (write once)	Poor fit given 1000:1 read:write ratio
Hybrid (push + pull threshold)	Cheap for the common case	Bounded, avoids write-amplification	Standard real-world answer

5. Common Interview Questions

Q1: "What breaks if you use pure fan-out-on-write for every account, including celebrities?"

Model approach: Name write-amplification explicitly — one post from a 50-million-follower account would trigger 50 million individual feed-store writes, an enormous, slow burden for a single post.

Q2: "Why not just use fan-out-on-read for everyone, avoiding the celebrity problem entirely?"

Model approach: Tie back to Step 2's estimated 1000:1 read:write ratio — making every read expensive to save write cost is the wrong trade when reads vastly outnumber writes; the hybrid model gets the best of both.

Q3: "How do you decide the follower-count threshold for switching an account to pull-based delivery?"

Model approach: Frame it as a tunable operational parameter, not a fixed law — set based on observed write-amplification cost at a given threshold, and note it can be monitored and adjusted (Phase 3, Ch 13) rather than hard-coded once and forgotten.

6. Practice Exercises

- 1 Run the full six-step framework for "design a news feed like Instagram," out loud, with a timer.
- 2 Calculate the exact write cost of a single post from an account with 10 million followers under pure fan-out-on-write, and compare it to the hybrid model's cost for the same post.
- 3 Design the merge step at read time: given precomputed pushed posts and live-pulled celebrity posts, how do you combine and rank them into one final feed?

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"System Design Interview Design Facebook News Feed"	Directly covers fan-out on write vs. read and the celebrity problem.
English	"System Design Mock Interview - Facebook News Feed"	Full live mock interview matching this chapter's exact structure.
Hindi	"High Level Design In Hindi: Design Facebook News Feed"	Dedicated Hindi-language walkthrough of this exact problem.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Bilingual course covering feed systems as part of the broader curriculum.

8. Key Takeaways

- The read:write ratio (here, over 1000:1) is the single number that justifies optimizing overwhelmingly for read performance.
- Fan-out on write (push) makes reads cheap but causes catastrophic write-amplification for high-follower-count accounts.
- Fan-out on read (pull) avoids write-amplification but makes every read expensive — a poor fit for a read-heavy system.
- The hybrid model — push for normal accounts, pull for accounts above a follower threshold — is the standard real-world answer.
- This same push/pull tension first appeared in Chapter 7's group chat fan-out and reappears in Chapter 9's Twitter design.
- Ranking (not strict chronology) is a real production concern worth naming even when out of scope for a full deep dive.
- Naming the specific failure mode of each pure approach, not just the hybrid's name, is what makes this deep dive convincing.